

Rapport de Projet de Module **Développement Mobile**

WasselTn

Application mobile Android de supervision
des livraisons en temps réel

Réalisé par : Talel Mejri
Encadré par : Mme Olfa Lammouchi
Classe : 2ING INFO B
Année universitaire : 2025 – 2026

Résumé Exécutif

WasselTn est une application mobile Android dédiée à la supervision des livraisons, développée dans le cadre du module *Développement Mobile* (2ème année Informatique — ENICarthage).

L'application prend en charge deux profils d'utilisateurs : le **contrôleur**, qui dispose d'une vue analytique globale, et le **livreur**, qui gère sa tournée quotidienne depuis son smartphone. Elle repose sur une architecture client-serveur : une application Android (Java / Retrofit2) consomme une API REST sécurisée construite avec **Laravel 10** et **Sanctum**, adossée à une base de données MySQL.

Les fonctionnalités clés incluent la gestion des états de livraison, un tableau de bord analytique, un système de messagerie contrôleur-livreur et l'intégration de la navigation GPS via Google Maps.

Table des matières

1	Description du Projet	4
1.1	Contexte et Problématique	4
1.2	Objectifs du Projet	5
1.3	Périmètre Fonctionnel	6
1.3.1	Fonctionnalités du Contrôleur	6
1.3.2	Fonctionnalités du Livreur	6
1.4	Stack Technique	7
2	Diagramme de Cas d’Utilisation	8
2.1	Acteurs du Système	8
2.2	Diagramme de Cas d’Utilisation	9
2.3	Description des Cas d’Utilisation Principaux	10
2.3.1	CU00 — S’authentifier	10
2.3.2	CU04 — Consulter le Tableau de Bord	10
2.3.3	CU08 — Modifier l’État d’une Livraison	10
3	Diagramme de Classes et Base de Données	12
3.1	Schéma de la Base de Données	12
3.2	Description des Tables	13
4	Architecture et API	14
4.1	Architecture Générale	14
4.2	Routes de l’API Laravel	14
4.2.1	Authentification et Gestion de Compte	14
4.2.2	Routes du Contrôleur (protégées par Sanctum)	15
4.2.3	Routes du Livreur (protégées par Sanctum)	15
4.3	Arbre de Navigation	16
5	Outils Utilisés et Difficultés Rencontrées	17
5.1	Environnement de Développement	17
5.1.1	Bibliothèques Android (Java)	18
5.2	Difficultés Rencontrées et Solutions	18
5.2.1	Gestion de l’Authentification et du Token Sanctum	18
5.2.2	Gestion des États de Livraison (Enum)	18

5.2.3	Synchronisation en Temps Réel	19
6	Interfaces de l'Application	20
6.1	Flux d'Authentification	20
6.2	Interfaces du Contrôleur	21
6.2.1	Liste des Livraisons sur une Période	23
6.2.2	Livraisons du Jour	23
6.2.3	Tableau de Bord	23
6.2.4	Messagerie Contrôleur → Livreur	23
6.3	Interfaces du Livreur	24
6.3.1	Tournée du Jour	24
6.3.2	Détail d'une Livraison	24
6.3.3	Modification de l'État	24
6.3.4	Messages d'Urgence	25
7	Annexe — Extraits de Code	26
7.1	Script SQL de Création (Extrait)	26
7.2	Backend Laravel — Authentification	27
7.3	Backend Laravel — Routes API	28
7.4	Android — Interface Retrofit2	29
7.5	Android — ServiceBuilder (Singleton Retrofit2)	30
7.6	Android — Adapteur RecyclerView	32

Chapitre 1

Description du Projet

1.1 Contexte et Problématique

Dans un contexte économique où la livraison à domicile connaît une croissance exponentielle, la gestion efficace des tournées représente un défi majeur pour les entreprises de distribution. La coordination entre les équipes de contrôle et les livreurs sur le terrain exige des outils modernes, réactifs et accessibles depuis des appareils mobiles.

Le projet **WasselTn** répond à ce besoin en proposant une application mobile Android dédiée à la **supervision des livraisons**, offrant simultanément aux **contrôleurs** une vision globale des opérations et aux **livreurs** les outils nécessaires à la gestion de leur tournée quotidienne.



Figure 1.1: Logo de l'application WasselTn

1.2 Objectifs du Projet

- ▶ Permettre aux **contrôleurs** de visualiser, filtrer et analyser l'ensemble des livraisons sur une période donnée.
- ▶ Offrir aux **livreurs** un accès rapide à leur tournée quotidienne et aux informations détaillées des clients.
- ▶ Faciliter la **communication** entre contrôleurs et livreurs via un système de messagerie intégré (messages d'information et d'urgence).
- ▶ Fournir un **tableau de bord analytique** pour le suivi des performances de livraison par livreur, par état et par client.
- ▶ Intégrer la **navigation GPS** (Google Maps) directement depuis la fiche de livraison.

1.3 Périmètre Fonctionnel

1.3.1 Fonctionnalités du Contrôleur

Contrôleur — Fonctionnalités principales

- ✓ Consulter la liste totale des livraisons sur une période donnée (état, livreur, date, numéro et montant de commande).
- ✓ Rechercher des livraisons par date de livraison ou par livreur.
- ✓ Consulter l'état des livraisons de la journée en cours avec tri par : état, livreur, client ou numéro de commande.
- ✓ Accéder à un tableau de bord avec statistiques par livreur, par état et par client.
- ✓ Consulter la liste des livreurs en tournée et leur envoyer des messages d'information.

1.3.2 Fonctionnalités du Livreur

Livreur — Fonctionnalités principales

- ✓ Consulter la liste ordonnée de ses livraisons du jour (ordre, numéro de commande, client, téléphone, ville).
- ✓ Accéder aux détails complets d'une livraison : contact client, adresse, lien Google Maps, articles, montant total et mode de paiement.
- ✓ Modifier l'état d'une livraison et ajouter des remarques en cas de non-livraison.
- ✓ Envoyer des messages d'urgence au contrôleur concernant une commande, avec inclusion automatique du numéro de commande.

1.4 Stack Technique

Composant	Technologie
Application Mobile	Android Studio Hedgehog/Iguana — Java 17
Backend / API REST	Laravel 10 (PHP 8.2)
Base de données	MySQL 8 — BDG_LivraisonCom_25
Authentification	Laravel Sanctum (tokens personnels)
Tests API	Postman
Client HTTP Android	Retrofit2 + OkHttp3
Sérialisation JSON	Gson
Stockage local	SharedPreferences
Cartographie	Google Maps via Android Intent
Maquettage	Figma
Versionnement	Git / GitHub

Chapitre 2

Diagramme de Cas d'Utilisation

2.1 Acteurs du Système

Le système **WasselTn** implique deux acteurs principaux :

- ▶ **Le Contrôleur** : responsable de la supervision globale des livraisons. Il accède à l'ensemble des données et peut communiquer avec tous les livreurs.
- ▶ **Le Livreur** : opérationnel sur le terrain. Il consulte exclusivement ses propres livraisons et peut signaler des incidents au contrôleur.

2.2 Diagramme de Cas d'Utilisation

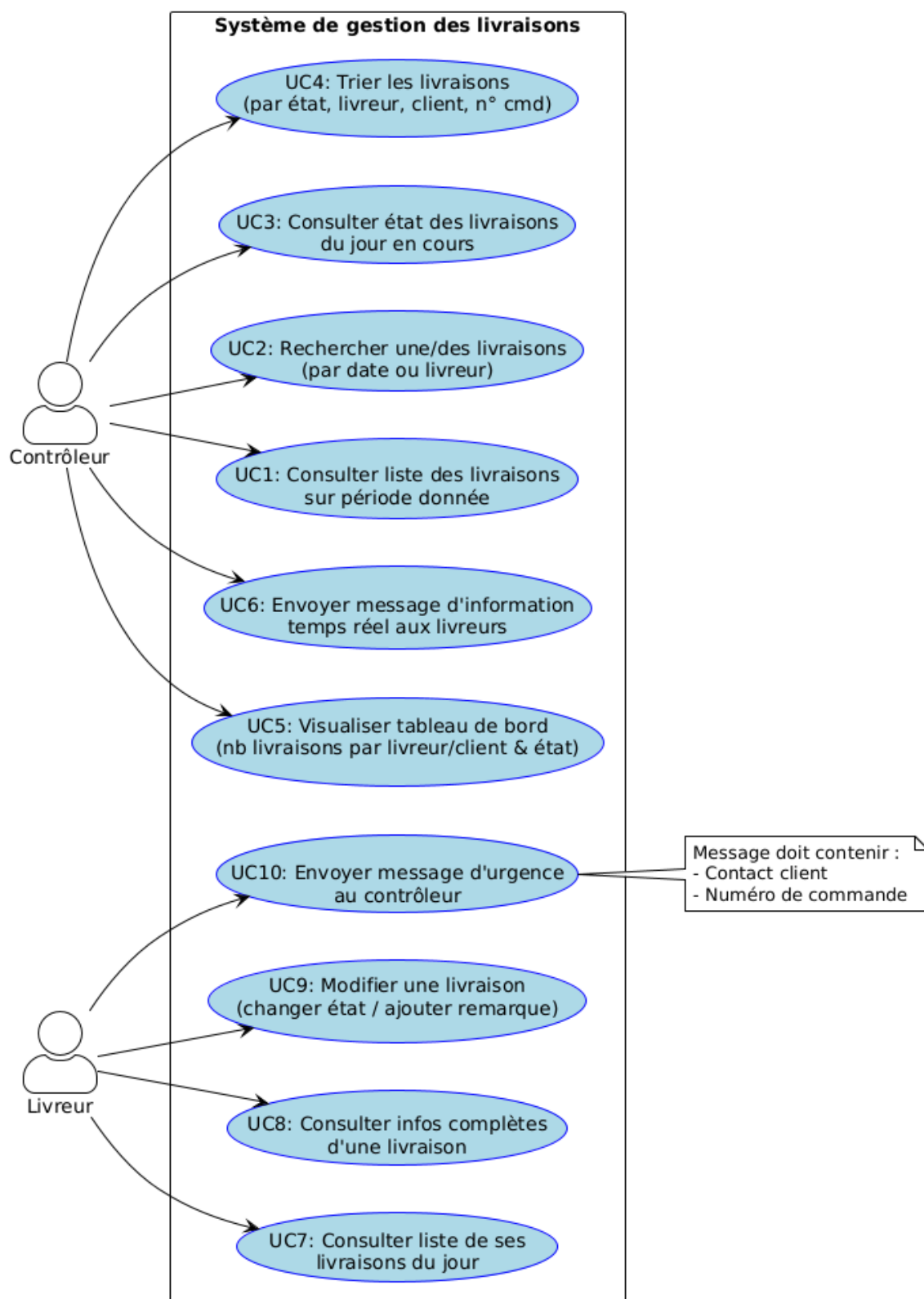


Figure 2.1: Diagramme de cas d'utilisation — Système WasselTn

2.3 Description des Cas d'Utilisation Principaux

2.3.1 CU00 — S'authentifier

Acteurs	Contrôleur, Livreur
Précondition	L'application est lancée
Scénario nominal	L'utilisateur saisit son Login et motP . Le système appelle POST /api/login . Laravel Sanctum vérifie les identifiants et retourne un <i>token</i> ainsi que le rôle de l'utilisateur.
Scénario alternatif	Identifiants incorrects : réponse HTTP 401, message d'erreur affiché à l'utilisateur.
Post-condition	Token stocké dans les SharedPreferences. Redirection vers l'interface correspondant au rôle.

2.3.2 CU04 — Consulter le Tableau de Bord

Acteurs	Contrôleur
Précondition	Le contrôleur est authentifié
Scénario nominal	L'application appelle GET /api/controleur/tableau-de-bord . Le système retourne les statistiques agrégées par livreur, par état et par client.
Post-condition	Le contrôleur dispose d'une vue synthétique des performances de livraison.

2.3.3 CU08 — Modifier l'État d'une Livraison

Acteurs	Livreur
Précondition	Le livreur est authentifié et a sélectionné une livraison
Scénario nominal	Le livreur choisit un nouvel état parmi $\{en_attente, en_cours, livrée, non_livrée, retard\}$. En cas de non-livraison, une remarque obligatoire est saisie. L'API est appelée pour effectuer la mise à jour.
Post-condition	L'état est mis à jour en base de données et devient instantanément visible pour le contrôleur.

Chapitre 3

Diagramme de Classes et Base de Données

3.1 Schéma de la Base de Données

Le diagramme ci-dessous présente le schéma réel de la base **BDG_LivraisonCom_25**, tel qu'implémenté sous MySQL Workbench. Il comprend les tables métier et les tables techniques ajoutées pour les besoins de l'application (**messages**, **password_reset_otps**).

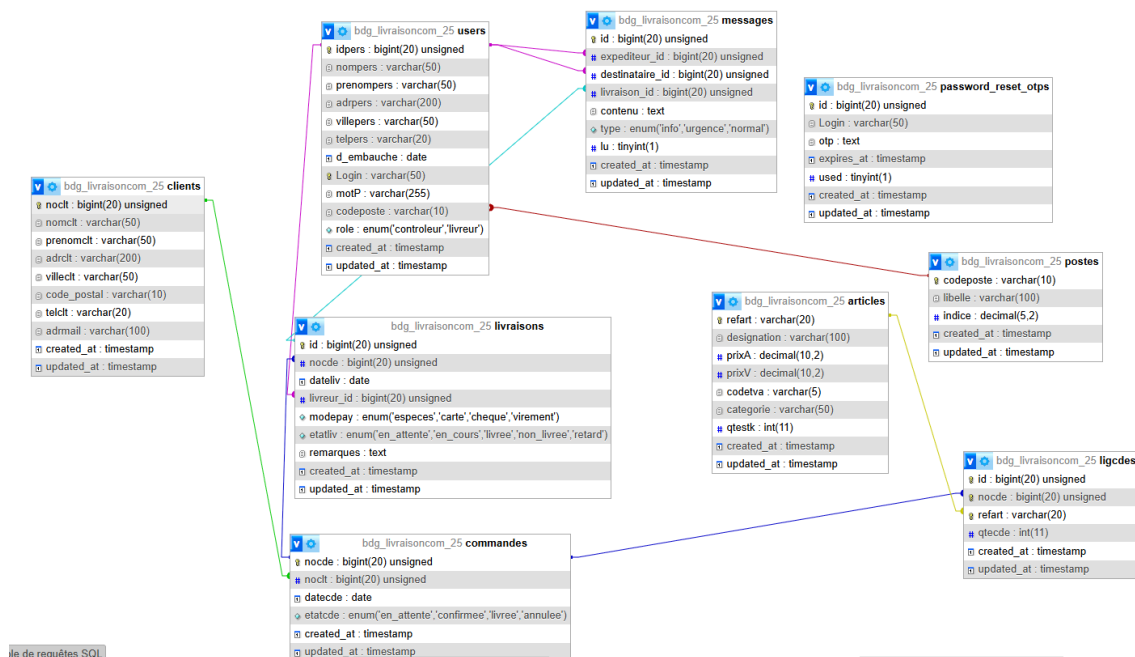


Figure 3.1: Schéma de la base de données — **BDG_LivraisonCom_25**

3.2 Description des Tables

Table	Description
users	Personnel de l'entreprise (livreurs et contrôleurs). Contient les identifiants de connexion, le rôle (<code>enum('contrôleur','livreur')</code>) et la clé étrangère vers postes .
clients	Coordonnées complètes des clients (nom, prénom, adresse, ville, code postal, téléphone, e-mail).
commandes	En-tête de commande liée à un client. État <code>enum</code> : <i>en_attente</i> / <i>confirmée</i> / <i>livrée</i> / <i>annulée</i> .
ligcdes	Lignes de détail de commande : article commandé et quantité.
articles	Catalogue produits avec prix d'achat/vente, TVA, catégorie, stock disponible.
livraisons	Suivi de la livraison : date, livreur assigné, mode de paiement (<code>enum espèces / carte / chèque / virement</code>), état (<i>en_attente</i> , <i>en_cours</i> , <i>livrée</i> , <i>non_livrée</i> , <i>retard</i>) et champ remarques .
postes	Référentiel des postes de travail (libellé, indice).
messages	Messages échangés entre contrôleur et livreurs. Contient l'expéditeur, le destinataire, la livraison concernée, le contenu, le type (<i>info</i> / <i>urgence</i> / <i>normal</i>) et un indicateur de lecture (<code>lu : tinyint</code>).
password_reset_otp	Gestion de la réinitialisation de mot de passe par code OTP (Login, otp, expires_at, used).

Chapitre 4

Architecture et API

4.1 Architecture Générale

L'application mobile Android communique avec un backend **Laravel 10** via une **API REST**. L'authentification est assurée par **Laravel Sanctum** (tokens personnels attachés à chaque utilisateur). Le client Android utilise **Retrofit2** pour consommer les endpoints et **Gson** pour la sérialisation JSON.

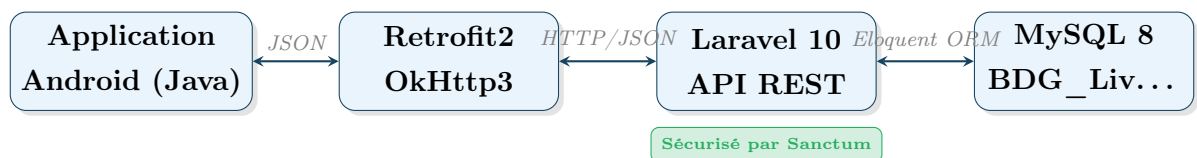


Figure 4.1: Architecture client-serveur de l'application WasselTh

4.2 Routes de l'API Laravel

4.2.1 Authentification et Gestion de Compte

Méthode	Endpoint	Description
POST	/api/login	Connexion et obtention du token Sanctum
POST	/api/forget-password	Demande de réinitialisation par OTP
POST	/api/reset-password	Réinitialisation du mot de passe avec OTP
GET	/api/me	Profil de l'utilisateur connecté

4.2.2 Routes du Contrôleur (protégées par Sanctum)

Méthode	Endpoint	Description
GET	controleur/livraisons/periode	Liste sur période (date_debut, date_fin)
GET	controleur/livraisons/rechercher	Recherche par date ou livreur_id
GET	controleur/livraisons/jour	Livraisons du jour (filtres + tri)
GET	controleur/tableau-de-bord	Statistiques globales
GET	controleur/livreurs-en-tournee	Liste des livreurs actifs du jour
POST	controleur/messages/envoyer	Envoyer un message à un livreur
GET	controleur/messages/envoyes	Historique des messages envoyés
PATCH	controleur/messages/{id}/lu	Marquer un message comme lu

4.2.3 Routes du Livreur (protégées par Sanctum)

Méthode	Endpoint	Description
GET	livreur/livraisons/jour	Tournée du jour (liste ordonnée)
GET	livreur/livraisons/{id}	Détails complets d'une livraison
PATCH	livreur/livraisons/{id}/etat	Modifier l'état + remarques
POST	livreur/messages/envoyer	Envoyer un message d'urgence
GET	livreur/messages/recus	Messages reçus du contrôleur

4.3 Arbre de Navigation

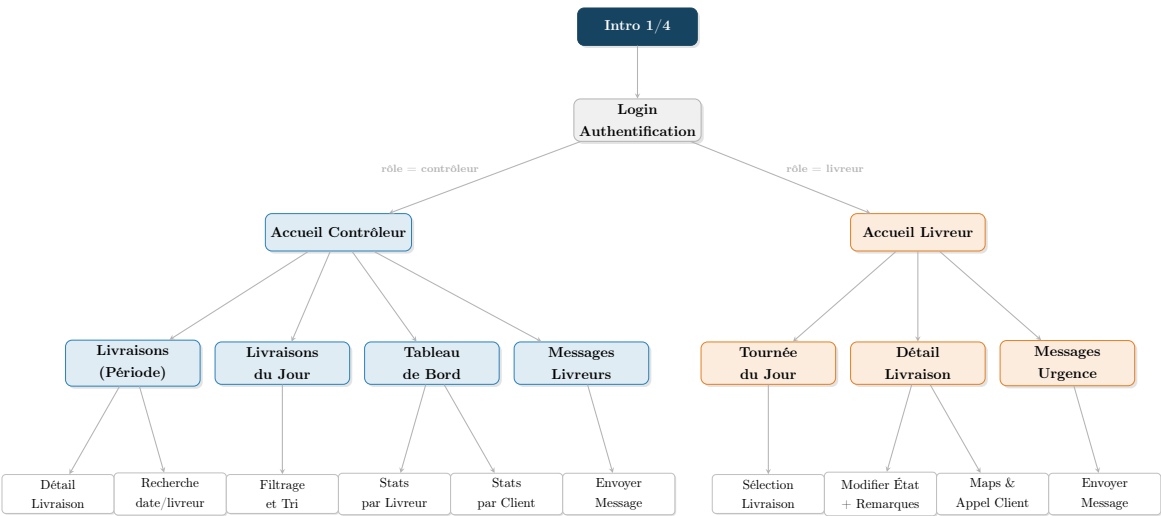


Figure 4.2: Arbre de navigation — Application WasselTn

Chapitre 5

Outils Utilisés et Difficultés Rencontrées

5.1 Environnement de Développement

Outil	Version	Rôle
Android Studio	Hedgehog/Iguana	IDE principal Android (Java)
Java	JDK 17	Langage de programmation mobile
Laravel	10.x	Framework backend PHP
PHP	8.2	Langage backend
MySQL	8.x	SGBDR
Postman	Dernière	Tests et documentation des end-points API
Git / GitHub	-	Versionnement du code source
Figma	-	Maquettage des interfaces

5.1.1 Bibliothèques Android (Java)

Bibliothèque	Utilisation
retrofit2	Client HTTP pour consommer l'API REST Laravel
okhttp3	Client HTTP sous-jacent à Retrofit2
gson	Sérialisation et désérialisation JSON
SharedPreferences	Stockage local du token Sanctum et des infos utilisateur
RecyclerView	Affichage performant des listes de livraisons
CardView	Mise en forme visuelle des cartes de livraison
Intent (Google Maps)	Navigation GPS vers l'adresse de livraison

5.2 Difficultés Rencontrées et Solutions

5.2.1 Gestion de l'Authentification et du Token Sanctum

L'intégration de Laravel Sanctum avec Android a nécessité une gestion rigoureuse du token dans les `SharedPreferences`. Chaque requête Retrofit doit inclure l'en-tête `Authorization: Bearer <token>`. La difficulté principale était de centraliser cet en-tête sans le dupliquer dans chaque appel.

Solution adoptée

Ajout d'un **intercepteur OkHttp** qui injecte automatiquement le token sur toutes les requêtes sortantes, configuré une seule fois dans le `ServiceBuilder`.

5.2.2 Gestion des États de Livraison (Enum)

L'enum `etatliv` comporte cinq valeurs avec des underscores. L'affichage dans l'adaptateur `RecyclerView` devait transformer ces valeurs en textes lisibles et associer une couleur sémantique à chaque état.

Solution adoptée

Implémentation d'une méthode `etatColor()` dans `LivraisonAdapter` utilisant un `switch` sur la valeur de l'enum et associant une couleur sémantique : `en cours` `livrée` `non livrée` `retard`.

5.2.3 Synchronisation en Temps Réel

Assurer que les modifications d'état effectuées par le livreur soient immédiatement visibles par le contrôleur a nécessité une stratégie de rechargement des données à chaque reprise de l'activité Android (`onResume`).

Solution adoptée

Déclenchement automatique d'un appel API lors du retour au premier plan (`onResume()`), garantissant la fraîcheur des données sans mise en place d'un mécanisme de *polling* complexe.

Chapitre 6

Interfaces de l'Application

6.1 Flux d'Authentification

L'application démarre sur un *splash screen* affichant le logo **WasselTn**, suivi d'un écran de bienvenue. L'écran de login présente deux champs (Login et Mot de passe) avec un bouton de connexion. Un lien *Mot de passe oublié ?* déclenche le flux de réinitialisation par code OTP. À la connexion réussie, l'application détecte le rôle et redirige vers l'interface correspondante.

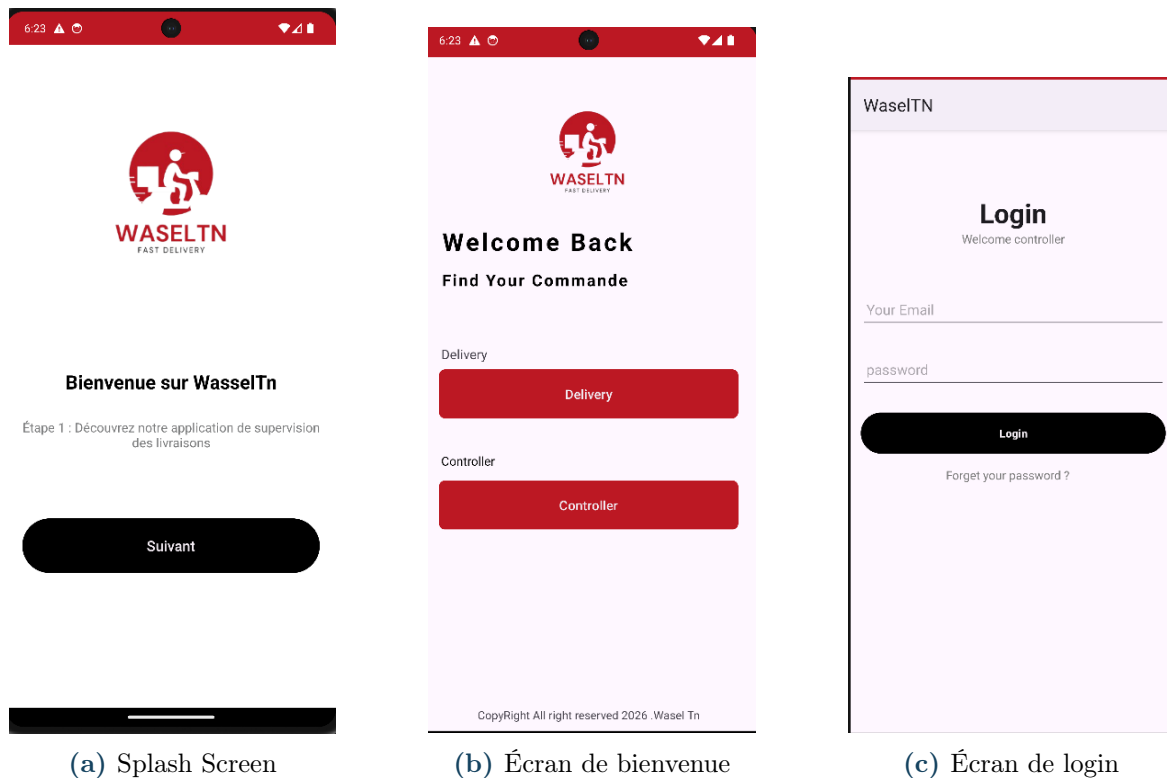


Figure 6.1: Écrans d'introduction et d'authentification

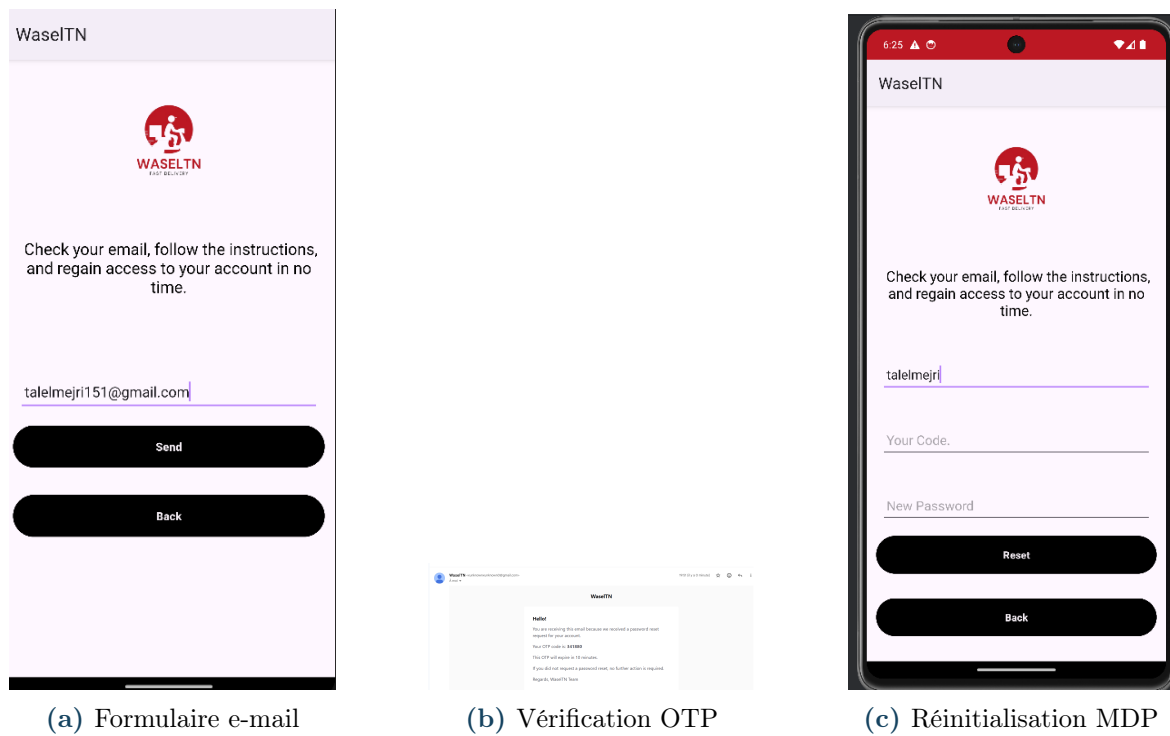
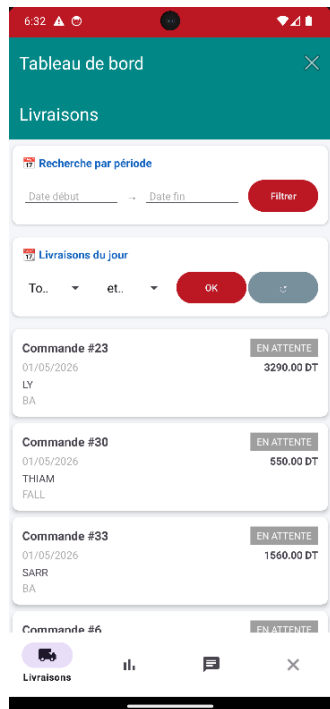


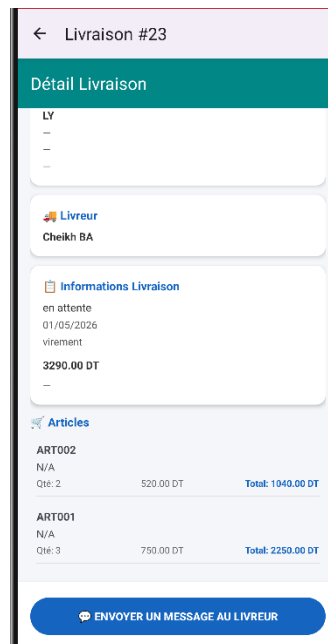
Figure 6.2: Flux de réinitialisation de mot de passe

6.2 Interfaces du Contrôleur

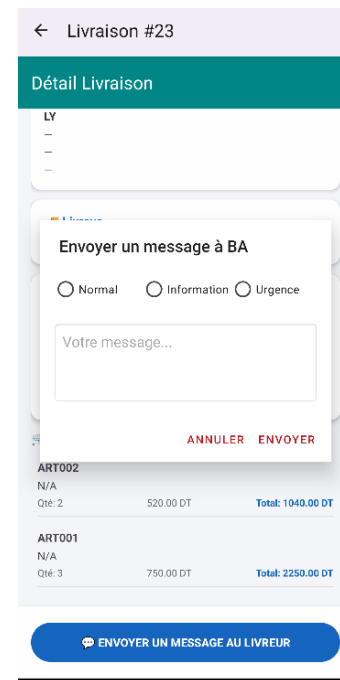
L'interface principale du contrôleur est organisée avec une *Bottom Navigation Bar* composée de quatre onglets : **Livraisons**, **Jour**, **Tableau de bord** et **Messages**.



(a) Tableau de bord

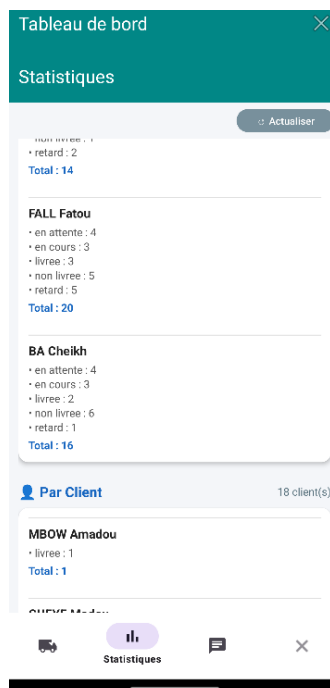


(b) Détails livraison

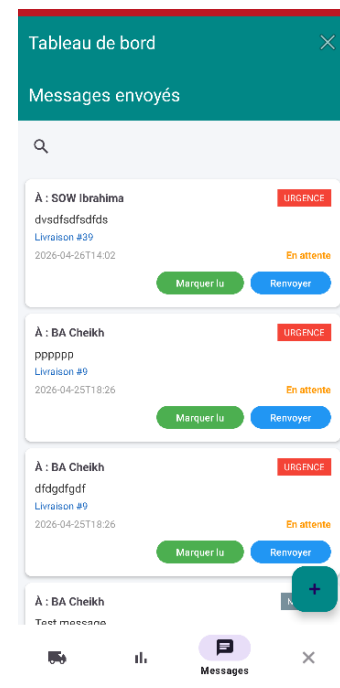


(c) Envoi de message

Figure 6.3: Interfaces du contrôleur — Tableau de bord et messagerie



(a) Statistiques



(b) Boîte de messages

Figure 6.4: Interfaces du contrôleur — Statistiques et messagerie

6.2.1 Liste des Livraisons sur une Période

L'utilisateur sélectionne une période via deux *date-pickers*. Les livraisons sont affichées dans un `RecyclerView` composé de cartes `CardView`. Chaque carte affiche : le numéro de commande, le client, le livreur, la date et l'état coloré en badge sémantique.

6.2.2 Livraisons du Jour

Cet écran présente les livraisons de la journée en cours avec des options de filtrage : *spinner* pour l'état, *spinner* pour le livreur, champ de recherche par numéro de client ou de commande, et menu de tri.

6.2.3 Tableau de Bord

Le tableau de bord affiche des statistiques agrégées : nombre de livraisons par livreur et par état, nombre de livraisons par client et par état.

6.2.4 Messagerie Contrôleur → Livreur

L'écran de messagerie liste les livreurs en tournée. Le contrôleur sélectionne un destinataire, rédige un message et l'envoie. L'historique des messages envoyés est accessible avec l'indicateur de lecture.

6.3 Interfaces du Livreur

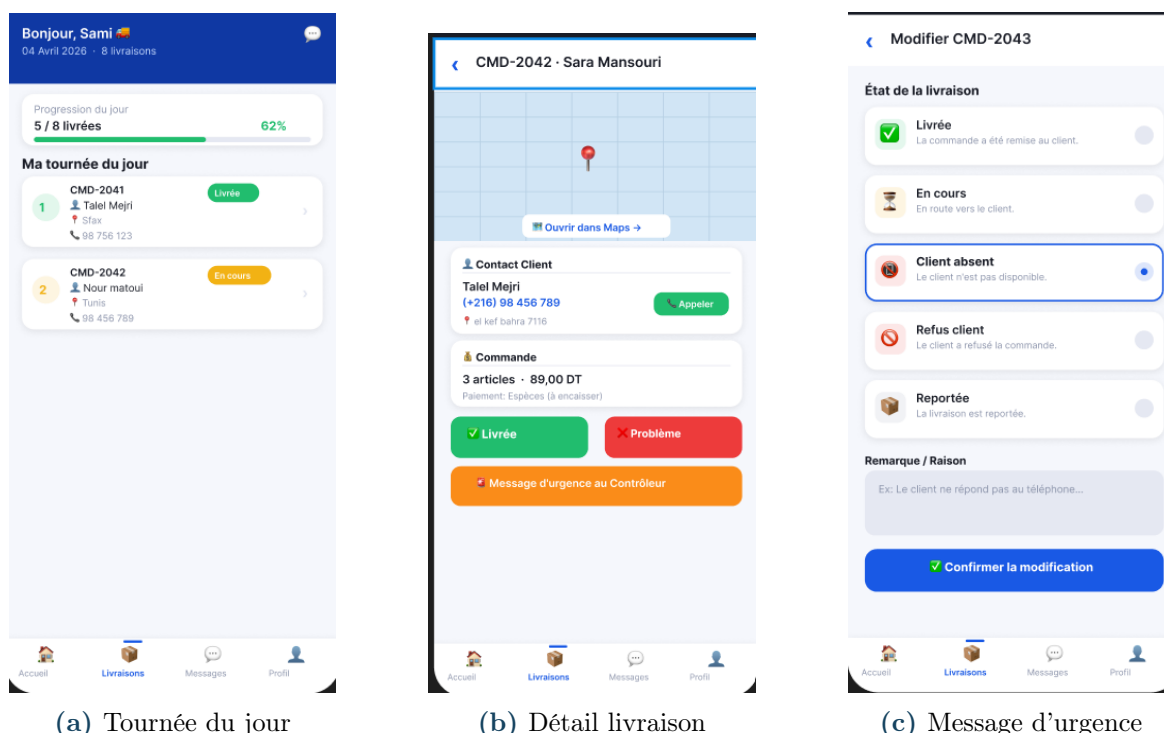


Figure 6.5: Interfaces du livreur

6.3.1 Tournée du Jour

L'écran principal du livreur affiche la liste ordonnée de ses livraisons du jour : ordre de passage, numéro de commande, nom du client, téléphone et ville.

6.3.2 Détail d'une Livraison

En sélectionnant une livraison, le livreur accède à une fiche complète : contact client avec bouton d'appel direct, adresse avec bouton *Ouvrir dans Maps* (Intent Google Maps), liste des articles commandés avec quantités, montant total et mode de paiement.

6.3.3 Modification de l'État

Le livreur modifie l'état via un *spinner* parmi les cinq valeurs de l'enum. En cas de non-livraison, un champ de remarque obligatoire apparaît pour saisir la raison (client absent, commande refusée, adresse incorrecte...).

6.3.4 Messages d'Urgence

Le livreur envoie un message d'urgence au contrôleur. Le message inclut automatiquement le numéro de commande et le contact du client concerné.

Chapitre 7

Annexe — Extraits de Code

7.1 Script SQL de Création (Extrait)

```
1 CREATE TABLE postes (
2     codeposte VARCHAR(10) PRIMARY KEY,
3     libelle VARCHAR(100),
4     indice DECIMAL(5,2),
5     created_at TIMESTAMP NULL,
6     updated_at TIMESTAMP NULL
7 );
8
9 CREATE TABLE users (
10     idpers BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
11     nompers VARCHAR(50),
12     prenompers VARCHAR(50),
13     adrpers VARCHAR(200),
14     villepers VARCHAR(50),
15     telpers VARCHAR(20),
16     d_embauche DATE,
17     Login VARCHAR(50) UNIQUE,
18     motP VARCHAR(255),
19     codeposte VARCHAR(10),
20     role ENUM('controleur','livreur'),
21     created_at TIMESTAMP NULL,
22     updated_at TIMESTAMP NULL,
23     FOREIGN KEY (codeposte) REFERENCES postes(codeposte)
24 );
25
26 CREATE TABLE livraisons (
27     id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
28     nocde BIGINT UNSIGNED UNIQUE,
29     dateliv DATE,
30     livreur_id BIGINT UNSIGNED,
31     modepay ENUM('especes','carte','cheque','virement'),
32     etatliv ENUM('en_attente','en_cours','livree','non_livree','retard'),
33     remarques TEXT,
```

```
34     created_at TIMESTAMP NULL,
35     updated_at TIMESTAMP NULL,
36     FOREIGN KEY (nocde) REFERENCES commandes(nocde),
37     FOREIGN KEY (livreur_id) REFERENCES users(idpers)
38 );
39
40 CREATE TABLE messages (
41     id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
42     expéditeur_id BIGINT UNSIGNED,
43     destinataire_id BIGINT UNSIGNED,
44     livraison_id BIGINT UNSIGNED,
45     contenu TEXT,
46     type ENUM('info','urgence','normal'),
47     lu TINYINT(1) DEFAULT 0,
48     created_at TIMESTAMP NULL,
49     updated_at TIMESTAMP NULL,
50     FOREIGN KEY (expéditeur_id) REFERENCES users(idpers),
51     FOREIGN KEY (destinataire_id) REFERENCES users(idpers),
52     FOREIGN KEY (livraison_id) REFERENCES livraisons(id)
53 );
```

Listing 7.1: Script SQL — Tables principales de BDG_LivraisonCom_25

7.2 Backend Laravel — Authentication

```
1 public function login(Request $request)
2 {
3     $request->validate([
4         'Login' => 'required|string',
5         'motP' => 'required|string',
6     ]);
7
8     $user = User::where('Login', $request->Login)->first();
9
10    if (!$user || !Hash::check($request->motP, $user->motP)) {
11        return response()->json([
12            'message' => 'Identifiants invalides'
13        ], 401);
14    }
15
16    $token = $user->createToken('api_token')->plainTextToken;
17
18    return response()->json([
19        'user' => $user->load('poste'),
```

```
20         'token' => $token ,
21         'role'  => $user->role ,
22     ] );
23 }
```

Listing 7.2: AuthController::login() — Authentification Laravel Sanctum

7.3 Backend Laravel — Routes API

```
1  // Routes publiques
2  Route::post('/login', [AuthController::class, 'login']);
3  Route::post('/forget-password', [AuthController::class, '
    forgetPassword']);
4  Route::post('/reset-password', [AuthController::class, '
    resetPassword']);
5  Route::get('/me', [AuthController::class, 'me']);
6
7  // Routes Controleur (protegees par Sanctum)
8  Route::prefix('controleur')->middleware('auth:sanctum')->group(
    function () {
9      Route::get('livraisons/periode', [ControleurController::
        class,
10         'livraisonsPeriode']);
11      Route::get('livraisons/rechercher', [ControleurController::
        class,
12         'rechercherLivraisons']);
13      Route::get('livraisons/jour', [ControleurController::
        class,
14         'livraisonsJour']);
15      Route::get('tableau-de-bord', [ControleurController::
        class,
16         'tableauDeBord']);
17      Route::get('livreurs-en-tournee', [ControleurController::
        class,
18         'livreursEnTournee']);
19      Route::post('messages/envoyer', [ControleurController::
        class,
20         'envoyerMessage']);
21      Route::get('messages/envoyes', [ControleurController::
        class,
22         'messagesEnvoyes']);
23      Route::patch('messages/{id}/lu', [ControleurController::
        class,
24         'marquerMessageLu']);
```

```
25 });
26
27 // Routes Livreur (protegees par Sanctum)
28 Route::prefix('livreur')->middleware('auth:sanctum')->group(
    function () {
29     Route::get('livraisons/jour', [LivreurController::class
30                                     ,
31                                     'livraisonsJour']);
32     Route::get('livraisons/{id}', [LivreurController::class
33                                     ,
34                                     'detailLivraison']);
35     Route::patch('livraisons/{id}/etat', [LivreurController::class
36                                             ,
37                                             'updateEtat']);
38     Route::post('messages/envoyer', [LivreurController::class
39                                       ,
36                                       'envoyerMessage']);
37     Route::get('messages/recus', [LivreurController::class
38                                   ,
39                                   'messagesRecus']);
39 });
```

Listing 7.3: routes/api.php — Routes de l'application WasselTn

7.4 Android — Interface Retrofit2

```
1 public interface RestApi {
2
3     @POST("login")
4     Call<UserResponse> loginUser(@Body Credentials payload);
5
6     @POST("forget-password")
7     Call<ResponseBody> forgotPassword(@Body Object data);
8
9     @POST("reset-password")
10    Call<ResponseBody> resetPassword(@Body Object data);
11
12    // -- Controleur --
13    @GET("controleur/livraisons/periode")
14    Call<LivraisonResponse> livraisonsPeriode(
15        @Query("date_debut") String dateDebut,
16        @Query("date_fin") String dateFin);
17
18    @GET("controleur/livraisons/jour")
```

```
19     Call<LivraisonResponse> livraisonsJour(  
20         @Query("etat")      String etat,  
21         @Query("livreur_id") Long livreurId,  
22         @Query("noclt")     Long noclt,  
23         @Query("nocde")     Long nocde,  
24         @Query("tri_par")   String triPar);  
25  
26     @GET("controleur/tableau-de-bord")  
27     Call<TableauBordResponse> tableauDeBord();  
28  
29     @GET("controleur/livreurs-en-tournee")  
30     Call<LivreursEnTourneeResponse> livreursEnTournee();  
31  
32     @POST("controleur/messages/envoyer")  
33     Call<SendMessageResponse> envoyerMessage(@Body MessageRequest  
34         body);  
35  
36     @PATCH("controleur/messages/{id}/lu")  
37     Call<ResponseBody> marquerMessageLu(@Path("id") long messageId)  
38         ;  
39  
40     // -- Livreur --  
41  
42     @GET("livreur/livraisons/jour")  
43     Call<LivraisonResponse> livraisonsJourLivreur();  
44  
45     @GET("livreur/livraisons/{id}")  
46     Call<LivraisonDetailResponse> detailLivraison(@Path("id") long  
47         id);  
48  
49     @PATCH("livreur/livraisons/{id}/etat")  
50     Call<ResponseBody> updateEtat(@Path("id") long id,  
51         @Body EtatRequest body);  
52  
53     @POST("livreur/messages/envoyer")  
54     Call<ResponseBody> envoyerMessageUrgence(@Body MessageRequest  
55         body);  
56 }
```

Listing 7.4: RestApi.java — Interface des appels API

7.5 Android — ServiceBuilder (Singleton Retrofit2)

```
1 public class ServiceBuilder {  
2
```

```
3     private static final String BASE_URL = "http://10.0.2.2:8000/
      api/";
4     private static Gson          gson;
5     private static Retrofit      retrofit;
6     private static RestApi       apiService;
7
8     private static OkHttpClient buildHttpClient(Context context) {
9         SharedUser sharedUser = new SharedUser(context);
10        UserResponse user      = sharedUser.getUser("user");
11
12        OkHttpClient.Builder builder = new OkHttpClient.Builder();
13
14        if (user != null && user.getToken() != null) {
15            final String token = user.getToken();
16            builder.addInterceptor(chain -> {
17                Request original = chain.request();
18                Request request  = original.newBuilder()
19                    .header("Authorization", "Bearer " + token)
20                    .header("Accept", "application/json")
21                    .method(original.method(), original.body())
22                    .build();
23                return chain.proceed(request);
24            });
25        }
26        return builder.build();
27    }
28
29    public static RestApi getApiService(Context context) {
30        if (gson == null)
31            gson = new GsonBuilder().setLenient().create();
32
33        retrofit = new Retrofit.Builder()
34            .baseUrl(BASE_URL)
35            .client(buildHttpClient(context))
36            .addConverterFactory(GsonConverterFactory.create(
37                gson))
38            .build();
39
40        apiService = retrofit.create(RestApi.class);
41        return apiService;
42    }
```

Listing 7.5: ServiceBuilder.java — Configuration Retrofit2 avec intercepteur

7.6 Android — Adapteur RecyclerView

```
1  public class LivraisonAdapter
2      extends RecyclerView.Adapter<LivraisonAdapter.ViewHolder> {
3
4      public interface OnLivraisonClickListener {
5          void onLivraisonClick(Livraison livraison);
6      }
7
8      private final List<Livraison>      livraisons;
9      private final OnLivraisonClickListener listener;
10
11     public LivraisonAdapter(List<Livraison> livraisons,
12                             OnLivraisonClickListener listener) {
13         this.livraisons = livraisons;
14         this.listener    = listener;
15     }
16
17     @NonNull @Override
18     public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent,
19                                         int vt) {
20         View v = LayoutInflater.from(parent.getContext())
21             .inflate(R.layout.item_livraison, parent, false);
22         return new ViewHolder(v);
23     }
24
25     @Override
26     public void onBindViewHolder(@NonNull ViewHolder h, int
27                                 position) {
28         Livraison liv = livraisons.get(position);
29         h.tvNocde.setText("Commande #" + liv.getNocde());
30         h.tvDate.setText(liv.getDateliv() != null ? liv.
31             getDateliv() : "-");
32         h.tvEtat.setText(liv.getEtatliv() != null
33             ? etatLabel(liv.getEtatliv()) : "-");
34         h.tvClient.setText(liv.getClient() != null
35             ? liv.getClient().getNom() : "-");
36         h.tvLivreur.setText(liv.getLivreur() != null
37             ? liv.getLivreur().getNompers() : "-");
38         h.tvMontant.setText(
39             String.format("%.2f DT", liv.getMontant_total()));
40
41         h.tvEtat.setBackgroundColor(etatColor(liv.getEtatliv()));
42         h.tvEtat.setTextColor(Color.WHITE);
43     }
44 }
```

```
40         h.card.setOnClickListener(v -> listener.onLivraisonClick(
41             liv));
42     }
43     @Override public int getItemCount() { return livraisons.size();
44     }
45     /** Couleur semantique par etat */
46     private int etatColor(String etat) {
47         if (etat == null) return Color.GRAY;
48         switch (etat) {
49             case "en_cours": return Color.parseColor("#2196F3");
50                 // Bleu
51             case "livree": return Color.parseColor("#4CAF50");
52                 // Vert
53             case "non_livree": return Color.parseColor("#F44336");
54                 // Rouge
55             case "retard": return Color.parseColor("#FF9800");
56                 // Orange
57             default: return Color.parseColor("#9E9E9E");
58                 // Gris
59         }
60     }
61
62     /** Libelle lisible par etat */
63     private String etatLabel(String etat) {
64         switch (etat) {
65             case "en_attente": return "En attente";
66             case "en_cours": return "En cours";
67             case "livree": return "Livree";
68             case "non_livree": return "Non livree";
69             case "retard": return "Retard";
70             default: return etat.replace("_", " ");
71         }
72     }
73
74     static class ViewHolder extends RecyclerView.ViewHolder {
75         CardView card;
76         TextView tvNocde, tvDate, tvEtat, tvClient, tvLivreur,
77             tvMontant;
78
79         ViewHolder(View v) {
80             super(v);
81             card = v.findViewById(R.id.cardLivraison);
82             tvNocde = v.findViewById(R.id.tvNocde);
83             tvDate = v.findViewById(R.id.tvDate);
```

```
78         tvEtat      = v.findViewById(R.id.tvEtat);
79         tvClient    = v.findViewById(R.id.tvClient);
80         tvLivreur   = v.findViewById(R.id.tvLivreur);
81         tvMontant   = v.findViewById(R.id.tvMontant);
82     }
83 }
84 }
```

Listing 7.6: LivraisonAdapter.java — Affichage de la liste des livraisons

Fin du Rapport

WasselTn — Application de Supervision des Livraisons

Talel Mejri | 2ING INFO B | 2025/2026

École Nationale d'Ingénieurs de Carthage
